REVIEW – III · FINAL REPORT

Final Demonstration and Innovation

GraphRAG-Powered Cyber Threat Intelligence Platform: A Multi-Model NoSQL Approach

Deliverable	Review – III (10 marks): final demonstration and innovation
Team	Shivansh Sharma (23BCE0254) Himali Sharma (23BCE0317) Aayush Sinha (23BCE0558)
Institution	VIT Vellore
Guide	Dr. Lydia Jane G
Repository	github.com/Shivansharmaaaa/graphrag
Date	October 2026

Deliverables covered

Complete working system — Sections 4–8; source in backend/ and frontend/

Performance evaluation — Section 9: SLOs, 1×–200× scalability, index ablations, ANN recall, load, LLM latency

Final report — this document

User manual — separate document: User_Manual_GraphRAG_CTI_Platform.pdf

Presentation and demo — slide deck and demo runbook in review3/

Contents

Section	Page
1. Abstract	3
2. Introduction	3
3. Background and Related Work	4
4. System Architecture	6
5. Data Model	7
6. Implementation	9
7. Innovation and Technical Complexity	12
8. Testing and Validation	13
9. Performance Evaluation	15
10. Requirement Traceability	22
11. Limitations and Future Work	22
12. Conclusion	23
References	24
Appendix A. Reproducing the Results	25

Figures (diagrams and charts) and **tables** are numbered in order of appearance. All measured values in Sections 8 and 9 are generated from the JSON result files in `review3/performance/`, produced by `make eval` and `make bench` on 2026-10-25.

1. Abstract

Cyber threat intelligence (CTI) is polyglot data. Relationships between threat actors, campaigns, malware and infrastructure form a graph; analyst reports are schema-variable documents; active indicators of compromise (IOCs) need millisecond lookups that expire on time; and analysts ask questions in natural language that need semantic recall. This project delivers a working CTI workstation that gives each workload its natural engine — Neo4j for the threat graph, MongoDB for reports, Redis for the IOC blocklist and Qdrant for report embeddings — and adds a GraphRAG layer that retrieves evidence from all four stores before a locally hosted LLM writes an answer in which every sentence must cite a graph edge, a passage, a report or an IOC state.

Review III completes and evaluates the system. The platform ingests STIX 2.1 bundles and free-text reports, enforces Traffic Light Protocol (TLP) clearance on every read path, serves a fail-closed enforcement API to firewalls and EDR tools, and records every answer for replay. On the six acceptance questions it recalls 100% of the required graph facts and source reports; with the default `llama3.2:3b` model the GraphRAG end-to-end p95 is **9.5 s** against a 15 s target (down from 52.3 s in September). A new benchmark harness scales the corpus from 1× to 200× (15,012 nodes, 2,400 reports) and ablates the indexes of each store. It exposed four scalability defects — linear entity linking, an unbounded traversal with unindexed node lookups, a whole-graph join on ingest and a thread-safety bug under concurrent load — whose fixes cut GraphRAG retrieval p95 at 200× from **14.7 s** to **185 ms** and keep every store-level query within its service level.

Keywords: cyber threat intelligence, polyglot persistence, NoSQL, graph database, vector search, retrieval-augmented generation, GraphRAG, performance evaluation.

2. Introduction

2.1 Problem

Security operations centres ask relational and semantic questions: which actors share command-and-control (C2) infrastructure, which campaigns reuse a commodity tool, which reports resemble a new phishing lure, and whether an indicator on the blocklist is still live. A single storage engine serves these workloads poorly [8], [9]: path queries become application-side joins in a document store, report bodies strain a graph store, and neither offers per-key expiry or approximate nearest-neighbour search. Retrieval-augmented generation (RAG) over CTI adds a second risk: a language model can produce fluent attributions that no stored evidence supports [21], [23].

2.2 Objectives

- **O1** Ingest STIX 2.1 bundles and free-text reports consistently into all four stores.

- **O2** Answer multi-hop graph questions (shared infrastructure, shared tooling, common victims) natively in Neo4j.
- **O3** Serve full, schema-variable reports from MongoDB with TLP-based access control.
- **O4** Maintain a hot IOC blocklist in Redis with per-key TTL and a fail-closed enforcement API.
- **O5** Answer natural-language questions with GraphRAG on a local LLM, with every sentence cited and auditable.
- **O6** Demonstrate that the design meets its latency targets and scales, with reproducible measurements.

2.3 What Review III adds

Review I fixed the design and Review II delivered the first working build. This review closes the remaining gaps recorded in the progress report and evaluates the system as the rubric asks: a complete system with an analyst console of eight pages and 35 API endpoints; a performance evaluation covering query latency, scalability, indexing and benchmarking; and the documentation set (this report, a user manual, slides and a demo runbook). The main engineering contributions of this review are:

- a **prompt budget** and model choice that bring GraphRAG p95 from 52.3 s to 9.5 s, meeting NFR2 on a laptop;
- a **groundedness-gate fix** that stops valid citations written in variant forms from being discarded;
- a **benchmark harness** (`make bench`) for 1×–200× corpus scaling, index ablations, ANN recall and API load, which found **four scalability defects**, all fixed and re-measured;
- **15 integration tests**, an evaluation gate and **continuous integration** that run the whole stack on every push.

3. Background and Related Work

Threat intelligence and sharing. Technical CTI — indicators, malware artefacts and procedures — is what automated defences consume, and its value lies in correlation rather than lookup [1], [2], [6]. STIX 2.1 already models CTI as a graph of objects and relationships [3], and ATT&CK gives a shared vocabulary of techniques [4]. MISP excels at disseminating events [5], [29] and OpenCTI offers a graph-backed product [16], but neither treats TTL-based enforcement, narrative similarity search and cited question answering as co-equal planes.

Polyglot persistence. “One size fits all” no longer holds for data management [8]; the NoSQL families each optimise a different access pattern and trade consistency guarantees accordingly [9]–[11]. Graph databases make multi-hop traversal a local operation through index-free adjacency [12], [26], and cybersecurity knowledge graphs have been built from reports and alerts [13]–[15].

Document stores keep heterogeneous reports without migrations [17]; key–value stores give constant-time lookups with native expiry [18]; and vector databases answer approximate k-NN queries with graph-based indexes such as HNSW [19], [20], [30].

From RAG to GraphRAG. RAG grounds generation in retrieved passages [21], [22], but passage retrieval alone cannot answer path questions. GraphRAG combines graph traversal with retrieval [23]–[25]; evaluation work stresses that answers must be faithful to their context [37] and that long, unprioritised contexts degrade model behaviour [36]. Benchmarks for serving systems [33], graph databases [34] and ANN indexes [35] motivate the measurement method in Section 9.

Capability	MISP	OpenCTI	Vanilla RAG	This platform
Multi-hop graph queries	limited	yes	no	yes (Neo4j)
Schema-variable reports	events	yes	passages	yes (MongoDB)
Per-key IOC expiry + enforcement API	no	no	no	yes (Redis, fail-closed)
Semantic search with payload filters	no	limited	yes	yes (Qdrant)
Cited natural-language answers	no	no	uncited	every sentence cited
Replay of an answer against the stores	no	no	no	yes
Runs fully offline (local LLM)	n/a	n/a	varies	yes

Table 1. Positioning against existing CTI platforms and plain RAG (Review I gaps G1–G5).

4. System Architecture

The implemented architecture matches the Review I design (Figure 1). A React console and downstream enforcement consumers are the only clients. A FastAPI service owns authentication, TLP policy, ingest, GraphRAG orchestration, audit and operations. Four NoSQL engines sit behind it, each with a closed set of operations, and a locally hosted LLM is called only by the GraphRAG orchestrator.

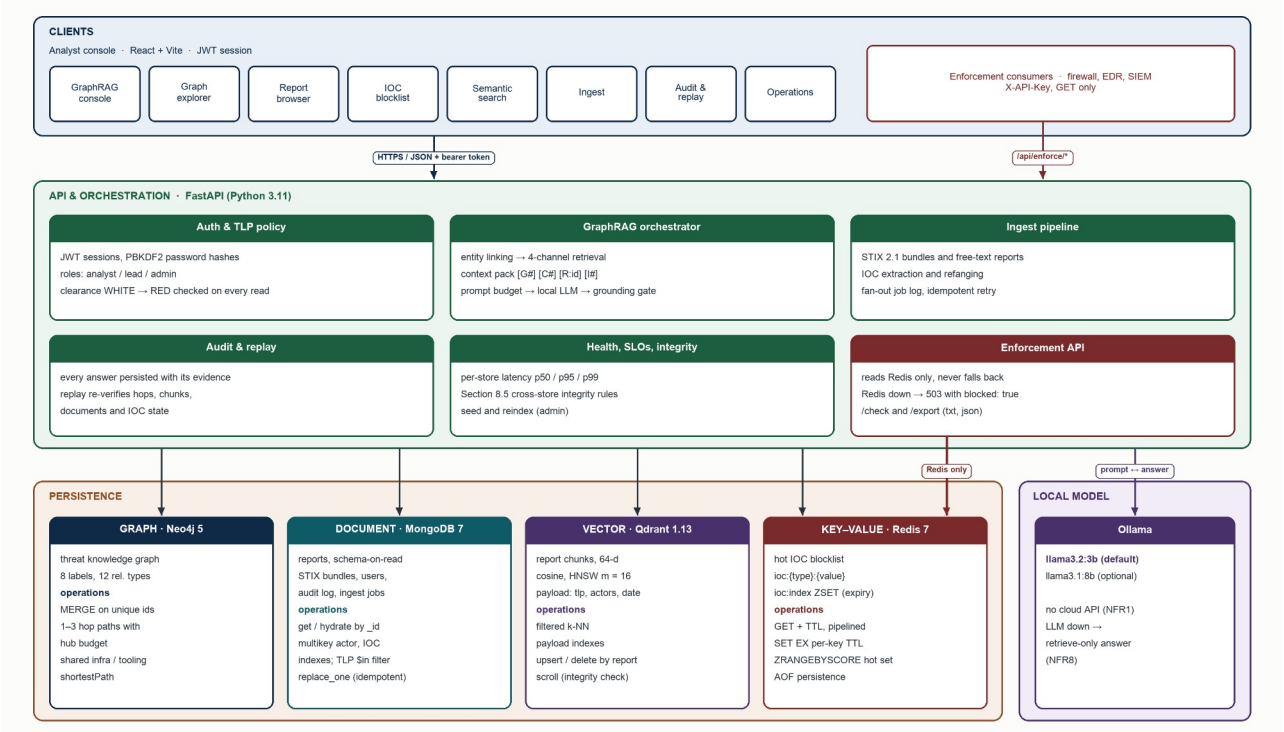


Figure 1. Implemented architecture: clients, API and orchestration, four NoSQL stores and the local model.

Store	Owns	Operations used	Why not another store
Neo4j 5	threat knowledge graph (8 labels, 12 relationship types)	MERGE by unique id; 1–3 hop paths with a hub budget; shortestPath	joins across documents are application code; Redis and Qdrant have no traversal
MongoDB 7	reports, STIX bundles, users, audit log, ingest jobs	get / hydrate by _id; multikey indexes on actors and IOCs; TLP $in filter	a graph node cannot hold report bodies with arbitrary nested fields
Redis 7	hot IOC blocklist	GET + TTL pipelined; SET EX; sorted-set expiry index	only Redis gives O(1) lookup with native per-key expiry
Qdrant 1.13	report chunk embeddings	filtered k-NN (HNSW, cosine); payload indexes	no other store offers ANN search with payload filters

Table 2. Responsibilities and closed operation set of each store.

Three keys join the stores (Figures 2 and 3): a report's `_id` in MongoDB is the `id` of its `Report` node and the `report_id` in every Qdrant point payload; and an indicator's `(type, value)` is both an `Indicator` node key and the Redis key `ioc:{type}:{value}`. These joins are verified by the integrity check in Section 5.3.

5. Data Model

5.1 Threat knowledge graph

Figure 2 shows the property graph as implemented. Identifiers are unique per label (constraints), commodity tools such as Mimikatz and Cobalt Strike are deliberately shared nodes, and `Indicator` nodes persist after their Redis key expires so that history and enforcement can disagree — the basis of use case UC4. The seed corpus loads 87 nodes and 162 relationships.

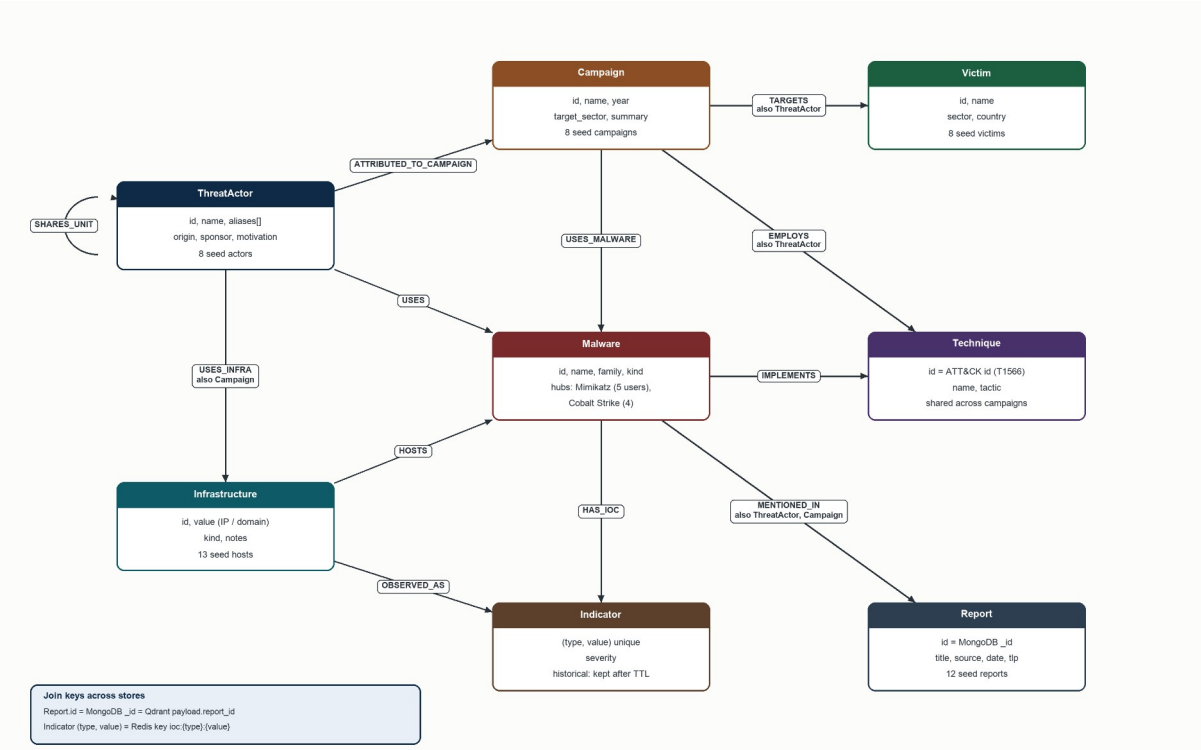


Figure 2. Graph schema (Neo4j): labels, key properties and relationship types.

5.2 Document, vector and key–value schemas

Reports share a small core envelope and diverge in report-specific fields (`attribution`, `build_pipeline`, `ics_protocols`), stored without migrations. Each report body is split into chunks of at most 420 characters; each chunk is a Qdrant point whose payload carries the fields that filters need. The Redis plane holds one string key per live IOC, expiring with `EX`, plus a sorted set scored by expiry time for the hot-set view.

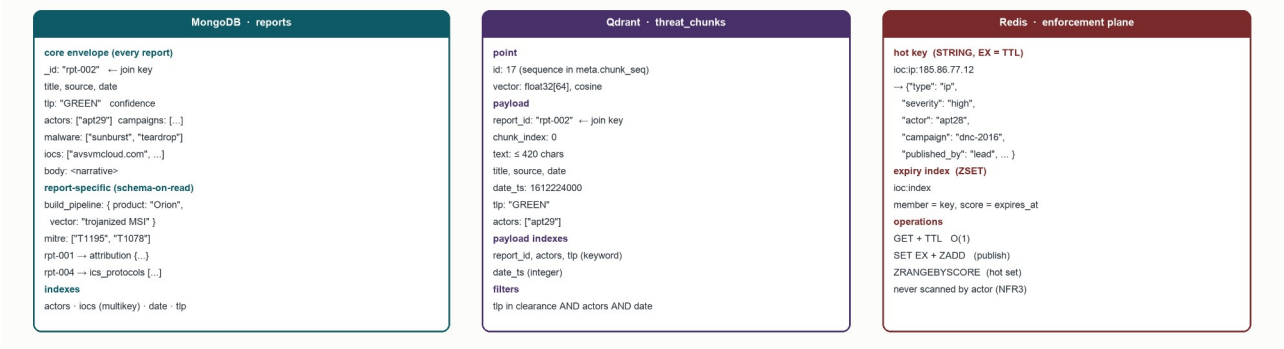


Figure 3. Document (MongoDB), vector (Qdrant) and key–value (Redis) schemas with their indexes.

5.3 Indexes, constraints and cross-store integrity

Store	Index or constraint	Serves
Neo4j	unique id on ThreatActor, Campaign, Malware, Infrastructure, Technique, Victim, Report; unique (type, value) on Indicator; range indexes on Infrastructure.value, Indicator.value, Indicator.id	MERGE during ingest; id lookups (index seek); IOC → graph lookups
MongoDB	reports: actors, iocs (multikey), date (desc), tlp; audit_log.ts; ingest_jobs.created_at; users.username (unique)	TLP-filtered listing; IOC history (UC4); newest-first views
Qdrant	HNSW (m = 16, ef_construct = 100); keyword payload indexes report_id, actors, tlp; integer date_ts	filtered k-NN; delete chunks of a report
Redis	key ioc:{type}:{value}; ZSET ioc:index	O(1) enforcement lookup; hot set and expiry listing

Table 3. Indexes and constraints (schema version 2, created at start-up by ensure_schema).

Integrity rule (Section 8.5 of Review I)	Kind	Effect if violated
Every MongoDB report has a Report node	hard	GraphRAG would cite a report with no graph context
Every Report node has a MongoDB document	hard	hydration fails
Every Qdrant point.report_id exists in MongoDB	hard	orphan chunks; GraphRAG refuses them
Every live Redis IOC traces to an Indicator node or a report	informational	operators may publish new IOCs (FR12)
Every MongoDB report has chunks in Qdrant	informational	report not reachable by semantic search

Table 4. Cross-store integrity rules checked by GET /api/admin/integrity and by the evaluation gate.

6. Implementation

6.1 Codebase

Part	Contents	Size
backend/app	FastAPI API, store clients, Cypher library, ingest, GraphRAG, security, metrics, seed corpus	3,520 lines of Python
backend/scripts	evaluation gate (evaluate.py) and benchmark harness (benchmark.py)	696 lines
backend/tests	16 unit tests and 15 integration tests	381 lines
frontend/src	React + Vite analyst console, 8 pages and 6 shared components	2,003 lines
deployment	docker-compose.yml (stores, app and llm profiles), docker-compose.bench.yml, Makefile, CI workflow	—

Table 5. Code organisation.

6.2 Ingest pipeline (FR1)

An ingest job turns a STIX bundle or a free-text report into a plan — graph nodes and edges, report documents and indicators — and executes it against Neo4j, MongoDB, Qdrant and optionally Redis in that order (Figure 4). STIX objects are merged into existing nodes by name or alias, relationship types are mapped onto the graph vocabulary (for example `attributed-to` is reversed into `ATTRIBUTED_TO_CAMPAIGN`), TLP comes from marking definitions, and indicator `valid_until` becomes the Redis TTL. Free text is linked to graph entities, and defanged IOCs such as `hxxp://evil[.]com` are refanged before extraction. There is no distributed transaction: each step is idempotent and recorded in `ingest_jobs`, so a failed job resumes from the first step that did not finish, and Qdrant refuses to index a document whose `Report` node is missing.

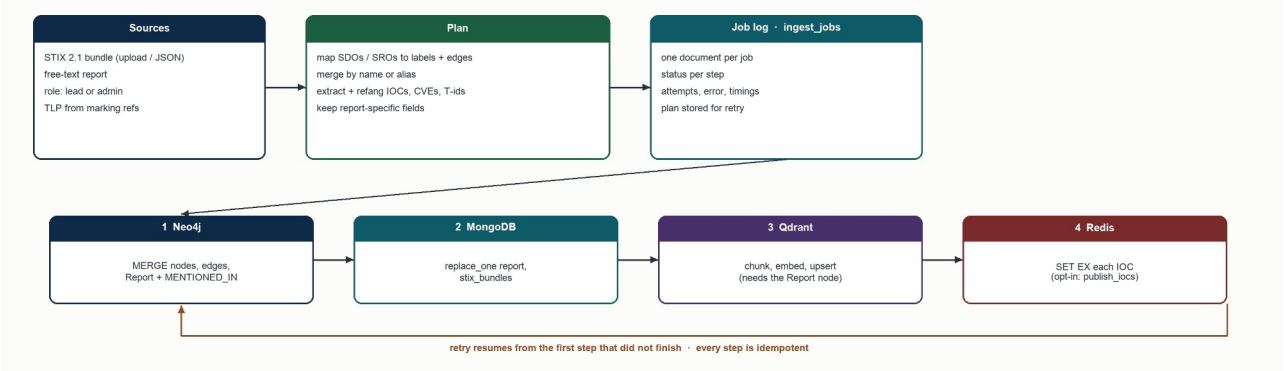


Figure 4. Ingest fan-out with a job log and idempotent retry.

6.3 GraphRAG orchestration (FR5, FR7, NFR4)

Figure 5 traces one question. Entity linking maps names, aliases and IOCs in the question to graph ids. Four channels then retrieve evidence: Qdrant k-NN over chunks with the analyst's TLP as a payload filter; Neo4j expansion of one to three hops from the linked entities, which may end at but never pass through a hub node (degree above 12), so Mimikatz cannot pull in the whole graph; MongoDB hydration of the cited reports, again TLP-checked; and Redis state for every IOC in the

evidence. Templated queries add the flagship findings (shared infrastructure, tool users) as citable edges.

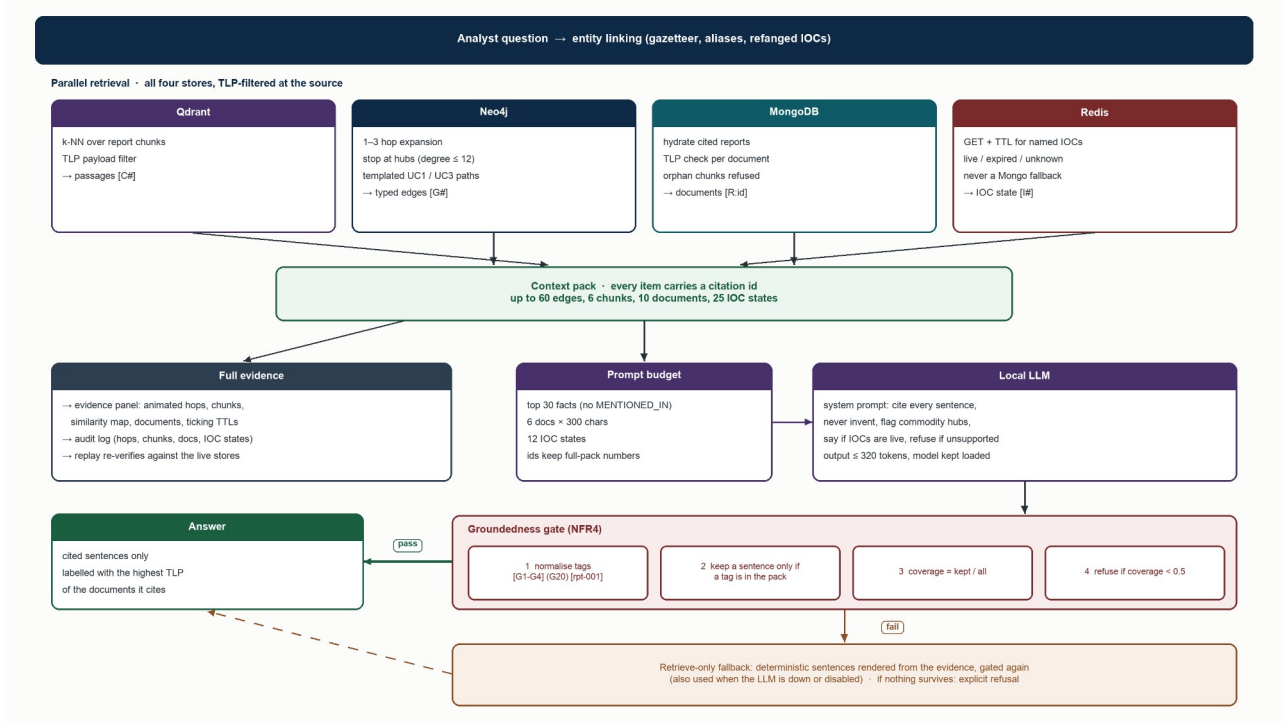


Figure 5. GraphRAG answer path: retrieval, context pack, prompt budget, local LLM and groundedness gate.

Every evidence item receives a citation id — `[G#]` edge, `[C#]` chunk, `[R:id]` report, `[I#]` IOC state. The full pack drives the evidence panel and the audit log, while the LLM receives a trimmed pack (Section 9.8). The groundedness gate splits the answer into sentences, normalises citation variants (`[G1-G4]`, `(G20)`, `[rpt-001]`), keeps a sentence only if one of its tags exists in the pack, and refuses the answer when fewer than half of the sentences survive. A refused LLM answer falls back to a deterministic retrieve-only answer rendered from the same evidence, which is gated again; the same path serves when the LLM is unreachable. The answer inherits the highest TLP of the documents it cites.

6.4 Security and access control (FR11, NFR3, NFR8)

Control	Implementation
Authentication	JWT (HS256, 8-hour expiry); PBKDF2-SHA256 password hashes with 200,000 iterations
Roles	analyst (query), lead (+ publish/revoke IOCs, ingest), admin (+ seed, reindex, delete reports, all audit entries)
TLP clearance	WHITE < GREEN < AMBER < RED (CLEAR → WHITE, AMBER+STRICT → AMBER, unknown → AMBER); enforced on report lists, report detail (403), semantic search, GraphRAG context, answer labels and audit visibility
Enforcement API	separate API-key surface for firewalls, EDR and SIEM; reads Redis only; a miss is never resolved from MongoDB; Redis down → HTTP 503 with blocked: true (fail closed)
Audit	every answer stores its hops, chunks, documents and IOC states; replay re-verifies each against the live stores

Control	Implementation
Data residency	local LLM through Ollama and local embeddings; no cloud API is called (NFR1)

Table 6. Security controls.

6.5 Analyst console

The console has eight pages: GraphRAG console, graph explorer, report browser, IOC blocklist, semantic search, ingest, audit and replay, and operations. Figure 6 shows the GraphRAG console answering acceptance question 4: citation chips link each sentence to its evidence, the commodity-hub caution appears beside the answer, and the evidence panel animates the cited hops. The user manual documents every page.

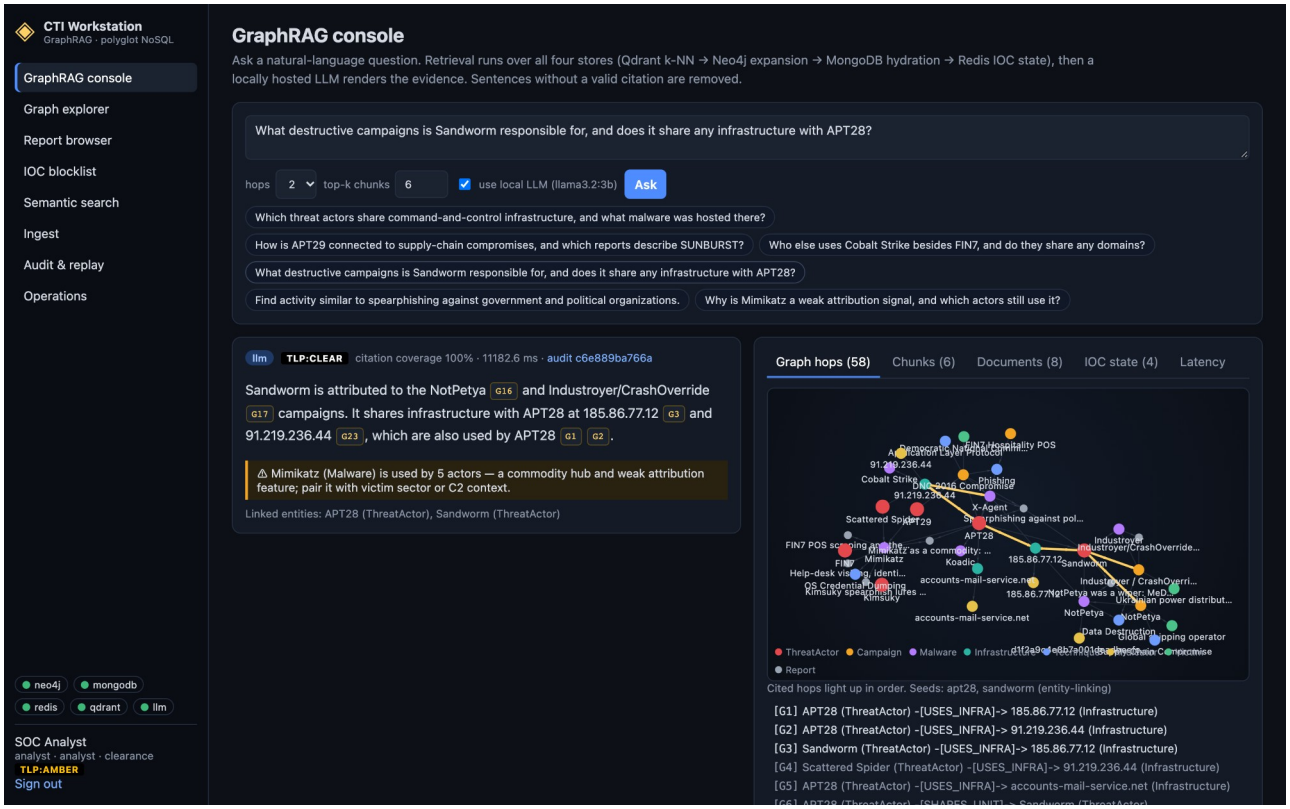


Figure 6. GraphRAG console: cited answer, hub caution and the evidence panel (llama3.2:3b, 100% citation coverage).

6.6 Deployment and operations

`docker compose up -d` starts the four stores; the `app` profile adds the API and an nginx-served console, and the `llm` profile an Ollama container. On first start the API creates the schema and seeds an empty stack. The operations page shows store health, per-store latency percentiles against their SLOs and the integrity check. Dumps of all four stores and a restore script support reproducibility (NFR5), and GitHub Actions runs the test tiers against fresh service containers on every push.

7. Innovation and Technical Complexity

7.1 Novel features

- **Four-store GraphRAG with typed citations.** Evidence from a graph, a document store, a vector index and a key–value cache is unified into one context pack where each item carries a store-specific citation id; the console resolves each id back to its store.
- **Groundedness gate with deterministic fallback.** Uncited sentences are removed, the answer is refused below 50% coverage, and a refused answer is replaced by evidence-only sentences rather than an unsupported one.
- **Hub-aware traversal.** Traversals stop at commodity hubs, bounding the evidence (60 edges at every corpus size in Section 9.3), and the answer must state that a commodity tool is a weak attribution signal (UC3).
- **TLP-aware retrieval and answer labelling.** Clearance is applied inside each store query — a Qdrant payload filter, a MongoDB $in, a graph filter — not after generation, and the answer is labelled with the highest TLP it cites.
- **Enforcement plane separate from the read path.** Consumers see only Redis, which fails closed, while the analyst read path degrades to retrieve-only answers when Redis or the LLM is down (NFR8).
- **Replayable answers.** An answer from weeks ago can be re-verified edge by edge, chunk by chunk and IOC by IOC against the current stores.
- **Local AI with a prompt budget.** The LLM sees a ranked, trimmed evidence pack sized for local inference, so a 3-billion-parameter model meets the interactive SLO without a GPU server.
- **Self-measuring system.** An evaluation gate and a benchmark harness are part of the code base; Section 9 shows they found and verified four performance fixes.

7.2 Advanced NoSQL capabilities used

Store	Capabilities beyond basic CRUD
Neo4j	variable-length path patterns with COUNT {} subquery predicates (hub budget); weighted path ranking with reduce(); composite (type, value) uniqueness; UNION-of-labels lookups that turn label-less matches into index seeks; CALL … IN TRANSACTIONS for batched bulk writes; PROFILE-driven tuning
MongoDB	schema-on-read documents with report-specific nested fields; multikey indexes on array fields; explain() plans used to verify index use; idempotent replace_one upserts; collStats for index sizing
Redis	per-key TTL with SET EX; sorted-set expiry index (ZADD, ZRANGEBYSCORE, ZCOUNT); pipelined GET + TTL batches; AOF persistence so TTLs survive restarts
Qdrant	HNSW with tunable search breadth (ef); keyword and integer payload indexes; filtered k-NN; filter-based deletes; exact search used as ground truth for recall

Table 7. Advanced capabilities of each engine used by the platform.

7.3 AI integration

The LLM is used only to render retrieved evidence as prose; it never decides what is retrieved and never writes to a store. It runs locally through Ollama (`llama3.2:3b` by default, `llama3.1:8b` optional [31], [32]) under a system prompt that requires a citation on every sentence, forbids invented entities and offensive content, and requires an exact refusal string when evidence is missing. Embeddings default to a deterministic 64-dimensional TF-IDF + truncated-SVD model, with a neural model (`nomic-embed-text`) available through configuration.

8. Testing and Validation

Tier	Count	What it checks
Unit tests	16	TLP ordering and clearance, password hashing, IOC extraction and refanging, entity linking (including equivalence of the indexed linker with the reference linker), STIX mapping, chunking, the groundedness gate and its citation forms
Integration tests	15	the real API against the live stores: auth and roles, TLP on lists, search and GraphRAG context, publish → enforce → revoke, fail-closed enforcement, LLM-down fallback, audit replay, ingest fan-out, delete cascade, integrity
Evaluation gate	6 questions	gold graph edges and reports, refusals, commodity caution, expiry probe, integrity; exits non-zero on any failure
Benchmarks	6 experiments	scalability, load, Redis, MongoDB and Neo4j index ablations, Qdrant recall (Section 9)
Continuous integration	every push	unit + integration + evaluation gate on fresh Neo4j, MongoDB, Redis and Qdrant containers; frontend build

Table 8. Test tiers. Integration tests remove everything they write, so they run safely against the demo stack.

8.1 Acceptance tests

The six Appendix A questions of Review I are the acceptance tests. Each must retrieve specified gold edges and gold reports. Table 9 shows the final run with the default model.

Question	Mode	Gold edges	Gold reports	Coverage	Latency
Which threat actors share command-and-control infrastructure, and what malware was hosted there?	LLM	2/2	1/1	0.50	7.8 s
How is APT29 connected to supply-chain compromises, and which reports describe SUNBURST?	retrieve-only (gate fallback)	1/1	1/1	1.00	9.5 s
Who else uses Cobalt Strike besides FIN7, and do they share any domains?	LLM	2/2	1/1	1.00	6.4 s
What destructive campaigns is Sandworm responsible for, and does it share any infrastructure with APT28?	LLM	2/2	1/1	1.00	6.1 s
Find activity similar to spearphishing against government and political organizations.	LLM	0/0	1/1	0.67	9.2 s
Why is Mimikatz a weak attribution signal, and which actors still use it?	LLM	2/2	1/1	0.60	9.4 s

Table 9. Acceptance results (2026-10-25, llama3.2:3b). Gold edge recall 1.00, gold report recall 1.00, mean citation coverage 0.79.

Every question retrieved all of its gold evidence and none was refused. In question 2 the model's draft fell below the coverage threshold and the gate replaced it with the deterministic evidence answer — the intended behaviour, not a failure. Question 6 carries the commodity-hub caution required by UC3. The same questions in retrieve-only mode, as run in CI, reach coverage 1.00 at about 50 ms per question.

9. Performance Evaluation

9.1 Method

Benchmarks run with `make bench` against a throwaway stack (`docker-compose.bench.yml`) on alternate ports, so synthetic data never reaches the demo stores; the demo stack is paused for the run. Each latency figure is a client-side percentile over repeated calls after warm-up (30–2,000 samples per measurement), taken through the same Python functions the API uses. To scale the corpus, the seed corpus is cloned 10×–200× through the production seed loader: clones receive new ids, names and infrastructure values from the RFC 2544 benchmarking range, one cross-clone infrastructure edge per actor, and they share the ATT&CK techniques and commodity tools, so hub degree grows with scale as it would in practice. Index experiments use dedicated synthetic collections. LLM measurements come from `make eval` on the demo stack.

Host	Apple M3 Pro, 18 GB RAM, macOS
Docker VM	2 vCPU, 3.8 GB shared by all containers
Engines	Neo4j 5.26.31 · MongoDB 7.0.43 · Redis 7.4.11 · Qdrant 1.13.4
Runtime	Python 3.11.7, FastAPI, one Uvicorn worker; Ollama on the host GPU (Metal)
Corpus scales	1× = 87 nodes / 12 reports; 10× = 762 nodes / 120 reports; 50× = 3,762 nodes / 600 reports; 100× = 7,512 nodes / 1,200 reports; 200× = 15,012 nodes / 2,400 reports

Table 10. Benchmark environment.

Threats to validity. All engines share two virtual CPUs, so absolute numbers are conservative for a server deployment but contention is real. The scaled corpus is synthetic and repetitive, which favours caches; the index experiments therefore use independent random data. Load is generated on the same host as the server.

9.2 Service-level objectives (NFR2)

Metric	Target	Measured	Result
Redis blocklist GET p99	< 5 ms	1.08 ms (500 calls); 0.534 ms at 500,000 keys	met
Neo4j 1 / 2 / 3-hop p95	< 500 ms	12.39 / 35.52 / 15.73 ms	met
Neo4j 2-hop p95 at 200×	< 500 ms	10.8 ms	met
GraphRAG end-to-end p95 (with LLM)	< 15 s	9.5 s (llama3.2:3b)	met
GraphRAG retrieval p95 at 200× (no LLM)	—	185 ms	within budget

Table 11. Service-level objectives.

9.3 Scalability from 1× to 200×

Scale	Nodes	Relationships	Reports	Chunks	IOCs	Load time (all stores)
1×	87	162	12	25	15	1.5 s
10×	762	1,665	120	250	150	4.0 s

Scale	Nodes	Relationships	Reports	Chunks	IOCs	Load time (all stores)
50×	3,762	8,345	600	1,250	750	15.8 s
100×	7,512	16,695	1,200	2,500	1,500	27.5 s
200×	15,012	33,395	2,400	5,000	3,000	54.5 s

Table 12. Corpus at each scale, loaded through the production seed path.

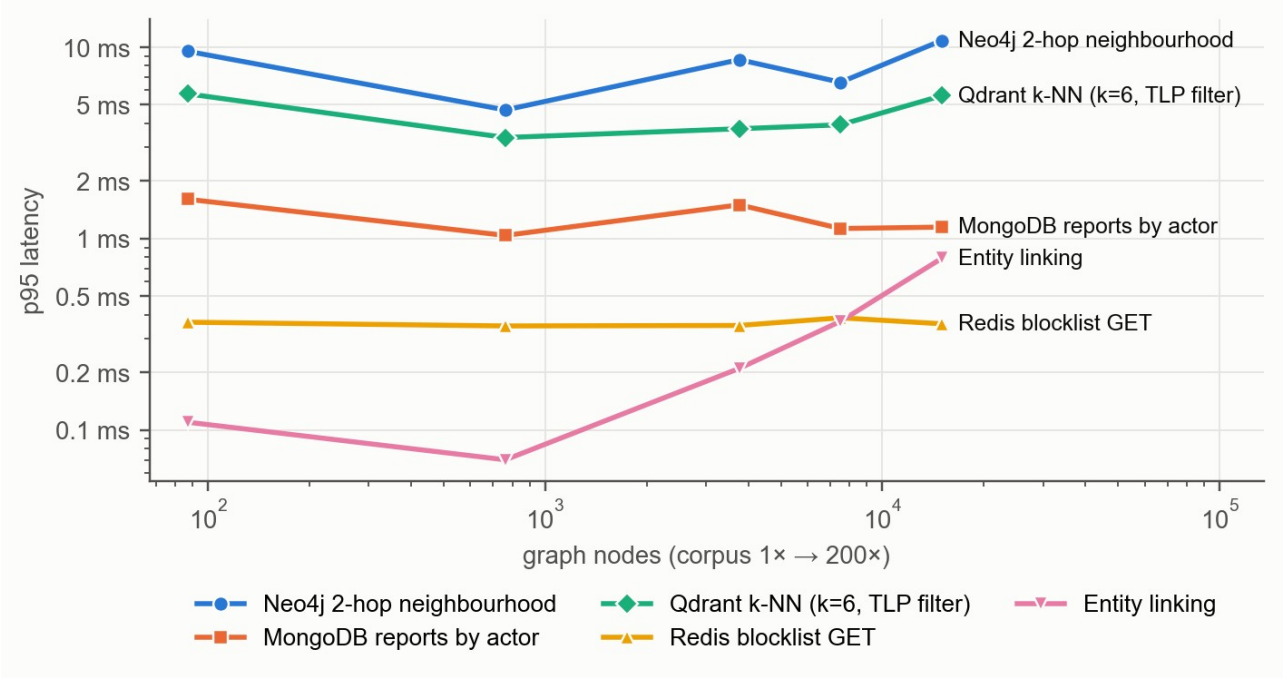


Figure 7. p95 latency of each store's GraphRAG query as the corpus grows 200× (after the fixes of Section 9.4).

Store-level latency is essentially flat across a 170-fold increase in graph size. A 2-hop neighbourhood stays near 9.51 ms–10.8 ms because traversal cost depends on the local neighbourhood, not on graph size (index-free adjacency), and the hub budget keeps the evidence at 60 edges and 10 documents at every scale. MongoDB reads by actor stay near 1.15 ms through the multikey index, Qdrant filtered k-NN near 5.58 ms, and Redis GET near 0.36 ms. Entity-linking p95 rises from 0.11 ms to 0.79 ms because a name such as APT28 has 200 near-duplicate clones to verify — still below a millisecond.

9.4 Bottlenecks found and fixed

The first scaling run did not look like Figure 7. GraphRAG retrieval grew roughly linearly with corpus size — median 26.8 ms at 1× to 675 ms at 200×, p95 14.7 s — and ingest throughput fell from 20.4 to 1.5 reports per second. Per-channel timing and Neo4j PROFILE output traced this to four defects, none visible on the 87-node seed corpus:

Defect	Cause	Fix
Linear entity linking	one regular-expression search per gazetteer surface form for every question and every ingest (≈30,000 at 200×)	inverted index from tokens and dotted runs to surface forms; candidates verified with the original word-boundary rule; a unit test proves identical results [39]

Defect	Cause	Fix
Unbounded global traversal + N+1 lookups	questions naming no actor listed every infrastructure-sharing pair, then fetched each missing node with a label-less MATCH (a full scan of all nodes, once per node)	LIMIT 25 ordered by shared-report evidence; one batched lookup via a UNION of labelled index seeks; an index on Indicator.id
Whole-graph join on ingest	every ingest re-linked OBSERVED_AS across all Infrastructure × Indicator pairs and invalidated the gazetteer	join restricted to the plan's values; gazetteer invalidated only when a job adds graph entities
Thread-unsafe vector client under load	one Qdrant REST client shared by FastAPI's worker threads: under 16 concurrent clients a socket was closed under another thread's read (HTTP 500)	one Qdrant client per worker thread; gazetteer refresh behind a lock

Table 13. Scalability defects exposed by the benchmark and their fixes.

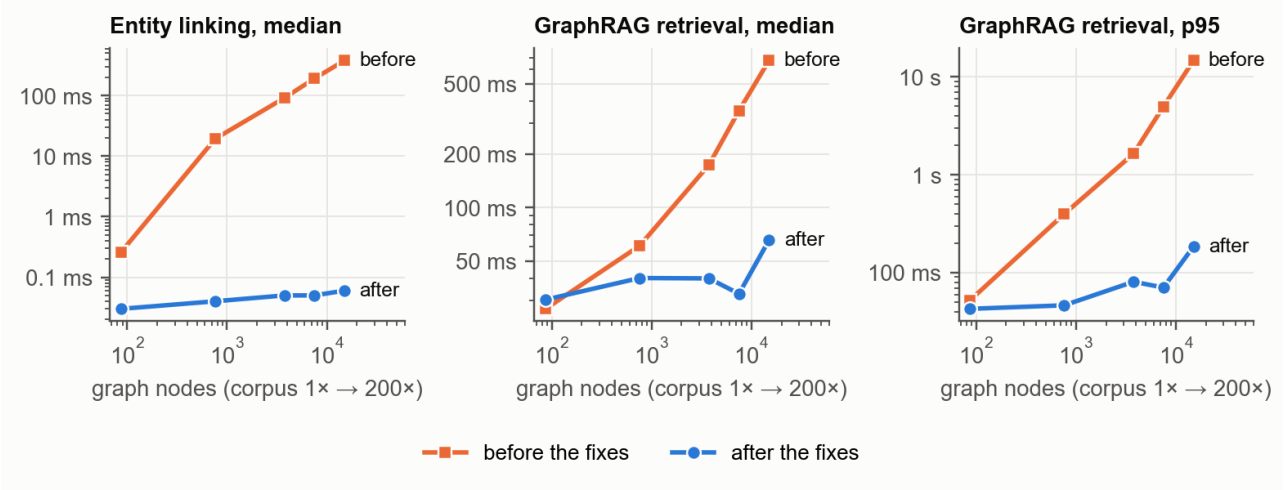


Figure 8. Entity linking and GraphRAG retrieval latency before and after the fixes, 1× to 200×.

At 200× the fixes reduce entity linking from 379 ms to 0.06 ms (median), GraphRAG retrieval from 675 ms to 65.4 ms (median) and from 14.7 s to 185 ms (p95), and raise ingest throughput from 1.5 to 31.6 reports per second (Figure 9). The full scale experiment now runs in 123.4 s instead of 331.8 s.

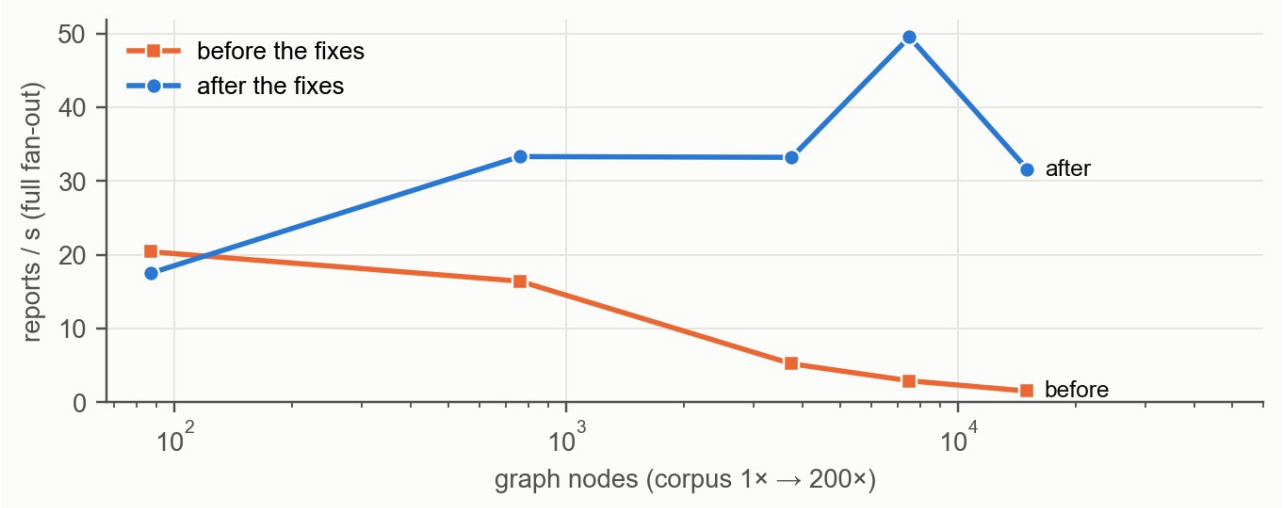


Figure 9. Ingest throughput through the full four-store fan-out, before and after the fixes.

9.5 Indexing

Each store's indexes were ablated on dedicated data: 200,000 synthetic reports in MongoDB and 100,000 hosts in Neo4j. Figure 10 and Table 14 show latency together with the work the engine reports — documents and keys examined from MongoDB `explain()`, database hits from Neo4j `PROFILE`.

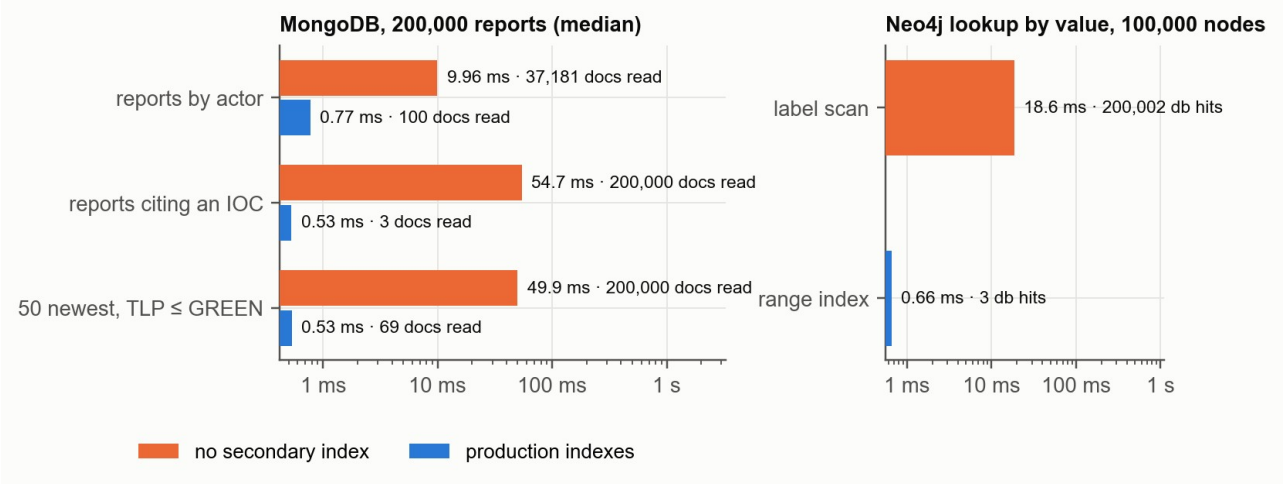


Figure 10. Index ablation: MongoDB report queries and Neo4j property lookup, without and with indexes (median latency).

Query	Plan without → with	Docs examined	Median latency	Speed-up
reports by actor (limit 100)	COLLSCAN → IXSCAN	37,181 → 100	9.96 ms → 0.77 ms	13×
reports citing an IOC	COLLSCAN → IXSCAN	200,000 → 3	54.7 ms → 0.53 ms	103×
50 newest at TLP ≤ GREEN	COLLSCAN → IXSCAN	200,000 → 69	49.9 ms → 0.53 ms	94×
Neo4j host by value	NodeByLabelScan → NodeIndexSeek	200,002 → 3 db hits	18.6 ms → 0.66 ms	28×

Table 14. Index ablation results.

Without secondary indexes MongoDB scans the collection (COLLSCAN) and reads up to every document; the multikey indexes reduce this to the matching documents. The newest-first TLP query reads 69 documents through the date index and 50 with a compound (tlp, date) index — a small further gain we record but do not need at this scale. The production indexes cost 14.0 MB for 44 MB of data and build in 0.95 s. In Neo4j, a label scan touches every node; the range index turns the lookup into a seek with 3 database hits.

9.6 Key–value enforcement plane

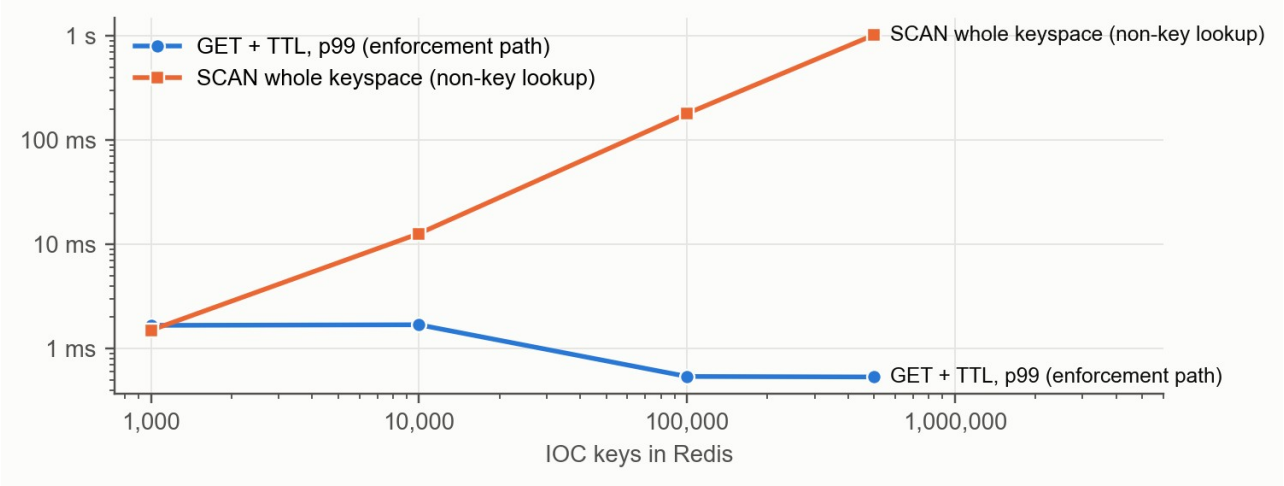


Figure 11. Redis: blocklist lookup p99 against a full keyspace scan as the number of IOC keys grows to 500,000.

The enforcement lookup (pipelined GET + TTL) stays at 0.29 ms median and 0.53 ms p99 at 500,000 keys, while any lookup that is not by key — such as finding IOCs by actor — needs a full SCAN that grows linearly to 1.0 s. This is why the design never queries Redis by attribute (NFR3) and keeps attribute queries in MongoDB and Neo4j. Memory grows to 191.6 MB at 500,000 keys (about 402 bytes per IOC including the expiry index).

9.7 Vector search: HNSW against exact search

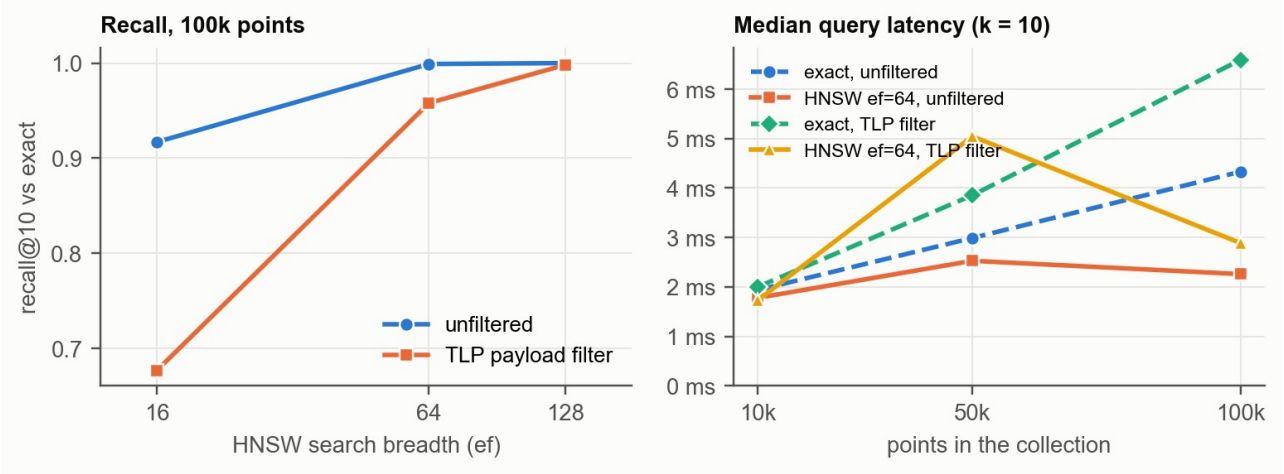


Figure 12. Qdrant: recall@10 of HNSW as search breadth grows, and median latency of HNSW and exact search as the collection grows.

Points	Exact (unfiltered)	HNSW ef = 64	Recall@10 ef = 64	Recall@10 ef = 128 (TLP filter)
10,000	1.94 ms	1.78 ms	1.000	1.000
50,000	2.99 ms	2.53 ms	1.000	1.000
100,000	4.33 ms	2.26 ms	0.999	0.998

Table 15. HNSW against exact search on 64-dimensional clustered vectors (100 queries, k = 10).

Exact search grows linearly with the collection (1.94 ms → 4.33 ms) while HNSW stays near 2.26 ms with recall@10 of 0.999 at ef = 64. Filtering by TLP needs a wider search: at 100,000 points recall

is 0.68 at ef = 16 but 0.998 at ef = 128. The platform's own collection (25 chunks, 5,000 at 200×) is below Qdrant's indexing threshold, so it is searched exactly — the optimal choice at that size.

9.8 Concurrency

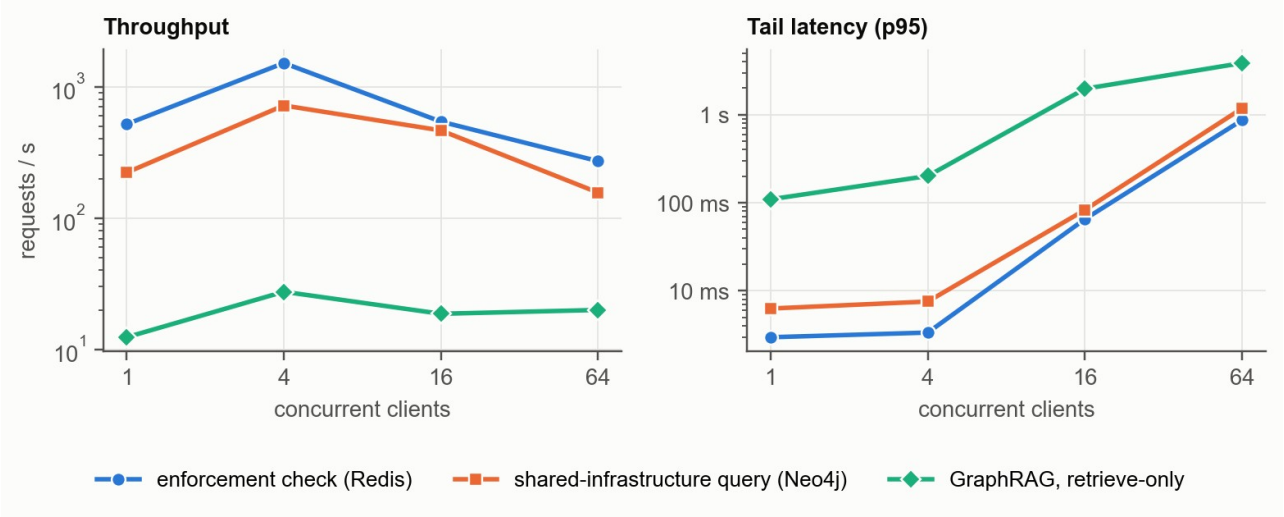


Figure 13. API throughput and p95 latency from 1 to 64 concurrent clients (200× corpus, one Uvicorn worker).

A single API worker peaks at 1,512 enforcement checks per second (4 clients) and 27.5 GraphRAG retrievals per second, with no errors at any level after the thread-safety fix. Beyond four clients throughput falls and tail latency rises sharply: the worker is bound by the Python interpreter lock and the two virtual CPUs it shares with the stores, so requests queue. Throughput therefore scales by adding Uvicorn workers or API replicas behind a load balancer — the stores are not the limit — and a deployment on dedicated cores would move the knee further out.

9.9 LLM latency

The GraphRAG SLO is the only one that depends on the language model. Profiling one call showed that with the full evidence pack (17,663 characters, 4,991 tokens) `llama3.1:8b` spent 17 s evaluating the prompt and 7.4 s generating 117 tokens. A prompt budget — the top 30 non-MENTIONED_IN facts, six documents of 300 characters and 12 IOC states, about half the characters — plus an output cap and keep-alive brought `llama3.1` to 21–27 s, and `llama3.2:3b` to 9.5 s p95 (Figure 14). Keeping the most relevant evidence also follows the observation that models use long contexts poorly [36].

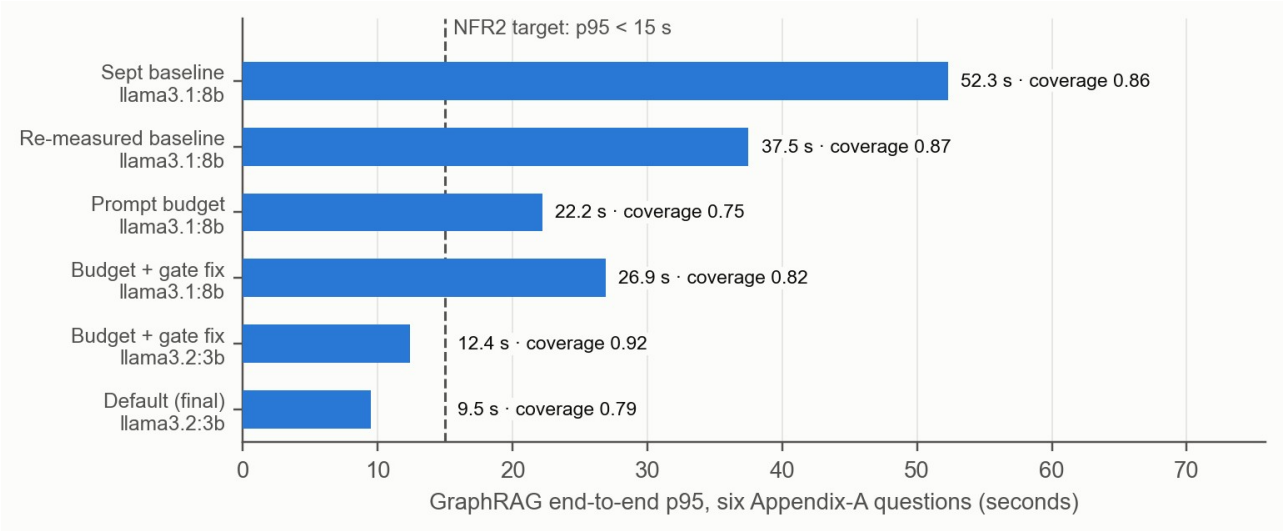


Figure 14. GraphRAG end-to-end p95 across measured configurations, with mean citation coverage (six acceptance questions).

The smaller model is the default because it meets the SLO with full gold recall; `llama3.1` gives fuller prose and remains one setting away (`OLLAMA_MODEL`). The groundedness-gate fix of this review matters here: before it, sentences such as "Sandworm is attributed to NotPetya (G20)" were discarded for using parentheses, lowering coverage for both models.

9.10 Summary

- Every NFR2 objective is met, including GraphRAG end-to-end p95 9.5 s < 15 s.
- Store-level queries stay flat from 87 to 15,012 nodes; GraphRAG retrieval p95 at 200× is 185 ms after the fixes.
- Indexes give 103× (MongoDB, IOC lookup) and 28× (Neo4j) speed-ups and cut examined records by up to five orders of magnitude.
- Redis lookups are independent of key count up to 500,000 keys; HNSW keeps recall@10 ≥ 0.999 at half the latency of exact search at 100,000 points.
- The benchmark found four defects invisible at seed scale; all are fixed and covered by tests or re-measurement.

10. Requirement Traceability

Req	Requirement	Implementation	Evidence
FR1	Ingest STIX 2.1 and free text, fan-out	ingest.py plans, job log, retry	unit + integration tests
FR2	Multi-hop graph queries	graph.py: shared infrastructure, tooling, victims, paths	acceptance Q1, Q3, Q4
FR3	O(1) blocklist with TTL, export	blocklist.py: SET EX, ZSET index; /enforce/*	Section 9.6; integration
FR4	Semantic search with filters	vectors.search: TLP, actor, date payload filters	integration; Section 9.7
FR5	GraphRAG Q&A on a local LLM	graphrag.ask, Ollama /api/chat	Table 9
FR6	Hydration of reports	documents.hydrate, GET /api/reports/{id}	integration
FR7	Evidence UI	GraphRAG evidence panel	Figure 6
FR8	Corpus and ongoing ingest	seed.py, data.py, auto-seed	CI first start
FR9	Health and SLOs	stores.health, metrics.py, Operations page	Section 9.2
FR10	Replay and audit	save_audit, replay	integration: hops re-verified
FR11	AuthN and TLP	security.py; TLP in every store query	integration (junior)
FR12	Publish and revoke IOCs	POST/DELETE /api/blocklist (lead)	integration
NFR1	Data residency	local Ollama and embeddings	no external calls
NFR2	Interactive latency	prompt budget, indexes, fixes of 9.4	Section 9
NFR3	Store isolation	enforcement reads Redis only	integration; 9.6
NFR4	Groundedness	groundedness gate, refusal, fallback	unit tests; Table 9
NFR5	Reproducibility	compose, pinned dependencies, versioned schema, dumps, CI	CI from empty stores
NFR6	Ethics and safety	public corpus, lab IOCs, system prompt	—
NFR7	Usability	UC1–UC4 inside the console	user manual, demo
NFR8	Availability	retrieve-only when LLM/Redis down; fail closed	integration tests

Table 16. Requirement traceability. All functional and non-functional requirements are met.

Gap	Review I observation	Closed by
G1	one engine forced to serve four workloads	four engines with closed operation sets (Table 2); measured in Section 9
G2	sharing platforms lack a query-native graph with enforcement state	Neo4j graph + Redis enforcement plane + cited Q&A
G3	knowledge graphs lack TTL cache and vectors	Redis and Qdrant beside Neo4j, joined by shared keys
G4	RAG over CTI is under-constrained	four-channel GraphRAG with a groundedness gate
G5	answers cannot be audited	evidence panel, audit log and replay

Table 17. Review I gaps and how the final system closes them.

11. Limitations and Future Work

- **Single-node deployment.** All stores run as single instances; clustering Neo4j, MongoDB replica sets and Redis replication are configuration work, not design changes.
- **One API worker.** Section 9.8 shows the knee at four concurrent clients on two virtual CPUs; production would run several workers or replicas.

- **Small, curated corpus.** Twelve public-knowledge reports cover every mechanism; the 200× scaling is synthetic. A TAXII feed connector would supply real volume.
- **Model quality versus latency.** `llama3.2:3b` meets the SLO but writes terser answers than `llama3.1:8b`; streaming the answer with the gate applied on completion would let the larger model feel interactive.
- **Embeddings.** The default TF-IDF + SVD model is deterministic and offline but weaker than neural embeddings; `nomic-embed-text` is supported and should become the default once download size is acceptable for the target environment.
- **Demo-grade secrets.** The JWT secret and consumer key in `.env.example` must be replaced outside a lab.

12. Conclusion

The platform shows that CTI is best served by a polyglot design in which each NoSQL model does what it is best at, and that GraphRAG over such a design can be both useful and accountable. Relationships live in Neo4j, reports in MongoDB, live indicators in Redis and narrative similarity in Qdrant; a local LLM turns their combined evidence into answers whose every sentence can be traced and replayed. Review III completed the system, verified it with 31 automated tests and continuous integration, and measured it: all latency objectives are met, store-level queries remain flat across a 200-fold corpus increase, and the benchmark harness proved its worth by exposing and confirming the fix of four defects that would have surfaced only in production.

References

[1] W. Tounsi and H. Rais, “A survey on technical threat intelligence in the age of information sharing,” Computers & Security, vol. 72, pp. 212–233, 2018.

[2] F. Skopik, G. Settanni, and R. Fiedler, “A problem shared is a problem halved: A survey on the dimensions of collective cyber defense through security information sharing,” Computers & Security, vol. 60, pp. 154–176, 2016.

[3] OASIS Open, “STIX Version 2.1,” OASIS Standard, June 2021. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/

[4] B. E. Strom et al., “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, 2020.

[5] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, “MISP: The design and implementation of a collaborative threat intelligence sharing platform,” in Proc. ACM WISCS, 2016, pp. 49–56.

[6] X. Liao et al., “Acing the IOC game: Toward automatic discovery and analysis of open-source cyber threat intelligence,” in Proc. ACM CCS, 2016, pp. 755–766.

[7] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, “TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of CTI sources,” in Proc. ACSAC, 2017, pp. 103–115.

[8] M. Stonebraker and U. Çetintemel, “‘One size fits all’: An idea whose time has come and gone,” in Proc. ICDE, 2005, pp. 2–11.

[9] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2012.

[10] R. Cattell, “Scalable SQL and NoSQL data stores,” ACM SIGMOD Record, vol. 39, no. 4, pp. 12–27, 2011.

[11] E. Brewer, “CAP twelve years later: How the ‘rules’ have changed,” Computer, vol. 45, no. 2, pp. 23–29, 2012.

[12] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. O’Reilly Media, 2015.

[13] S. Noel et al., “CyGraph: Graph-based analytics and visualization for cybersecurity,” in Handbook of Statistics, vol. 35. Elsevier, 2016, pp. 117–167.

[14] A. Piplai, S. Mittal, A. Joshi, T. Finin, J. Holt, and R. Zak, “Creating cybersecurity knowledge graphs from malware after action reports,” IEEE Access, vol. 8, pp. 211691–211703, 2020.

[15] I. Sarhan and M. Spruit, “Open-CyKG: An open cyber threat intelligence knowledge graph,” Knowledge-Based Systems, vol. 227, 2021.

[16] Filigran, “OpenCTI documentation,” 2024. [Online]. Available: https://docs.opencti.io/

[17] MongoDB, Inc., “MongoDB manual (document model, indexes, explain),” 2024.

[18] Redis Ltd., “Redis documentation: EXPIRE, sorted sets and pipelining,” 2024.

[19] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.

[20] Qdrant, “Qdrant documentation: collections, HNSW and payload filtering,” 2024.

[21] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.

[22] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in Proc. EMNLP, 2020, pp. 6769–6781.

[23] D. Edge et al., “From local to global: A graph RAG approach to query-focused summarization,” arXiv:2404.16130, 2024.

[24] B. Peng et al., “Graph retrieval-augmented generation: A survey,” arXiv:2408.08921, 2024.

[25] M. Yasunaga, H. Ren, A. Bosselut, P. Liang, and J. Leskovec, “QA-GNN: Reasoning with language models and knowledge graphs for question answering,” in Proc. NAACL, 2021.

[26] Neo4j, Inc., “Neo4j Cypher manual (indexes, PROFILE, subqueries),” 2024.

[27] ENISA, “ENISA threat landscape,” European Union Agency for Cybersecurity, 2024.

[28] Mandiant, “M-Trends,” Google Cloud, 2024.

[29] CIRCL / MISP Project, “MISP data models and objects,” 2024. [Online]. Available: https://www.misp-project.org/

[30] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[31] A. Grattafiori et al., “The Llama 3 herd of models,” arXiv:2407.21783, 2024.

[32] Ollama, “Ollama documentation and REST API,” 2024. [Online]. Available: https://github.com/ollama/ollama

[33] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in Proc. ACM SoCC, 2010, pp. 143–154.

[34] O. Erling et al., “The LDBC Social Network Benchmark: Interactive workload,” in Proc. ACM SIGMOD, 2015, pp. 619–630.

[35] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms,” Information Systems, vol. 87, 2020.

[36] N. F. Liu et al., “Lost in the middle: How language models use long contexts,” Trans. Assoc. Comput. Linguistics, vol. 12, pp. 157–173, 2024.

[37] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” in Proc. EACL (System Demonstrations), 2024.

[38] FIRST, “Traffic Light Protocol (TLP) version 2.0,” 2022. [Online]. Available: https://www.first.org/tlp/

[39] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” Commun. ACM, vol. 18, no. 6, pp. 333–340, 1975.

Appendix A. Reproducing the Results

All results in this report can be regenerated from the repository. The demo stack and the benchmark stack use different ports and volumes.

```
make infra && make install          # four stores, Python and Node dependencies
make api                            # FastAPI on :8000 (seeds empty stores)
make web                            # console on :5173
ollama pull llama3.2:3b             # local model (default)
make test                           # unit tests
make test-integration               # API against the live stores
make eval-check                     # acceptance questions + gates  ->
backend/eval_report.json
docker compose --profile app stop   # pause the demo stack before benchmarking
make bench                          # throwaway stack ->
review3/performance/benchmark_results.json
python review3/make_charts.py       # charts       (pip install matplotlib==3.9.2)
python review3/generate_figures.py  # diagrams     (pip install pillow)
node review3/generate_report.js     # this report  (npm install in review3/,
LibreOffice for PDF)
```

The raw results are in `review3/performance/`: `benchmark_results.json` (after the fixes), `benchmark_baseline_before_fixes.json`, the three evaluation runs and `llm_latency_runs.json`, which records the provenance of every LLM measurement in Figure 14.